

Digital Signature Standard

Материал из Википедии — свободной энциклопедии

DSS (*Digital Signature Standard*) — американский стандарт, описывающий Digital Signature Algorithm (DSA), который может быть использован для генерации цифровой подписи. Цифровая подпись служит для установления изменений данных и для установления подлинности подписавшейся стороны. Получатель подписанных данных может использовать цифровую подпись для доказательства третьей стороне факта, что подпись действительно сделана отправляющей стороной.

DSS, Digital Signature Standard	
Создатель	<u>Национальный институт стандартов и технологий</u>
Создан	август <u>1991</u>
Размер ключа	512—1024 бит
Размер подписи	два числа по 160 бит

Содержание

<u>Введение</u>
<u>Использование DSA</u>
<u>Параметры DSA</u>
<u>Генерация подписи</u>
<u>Проверка подписи</u>
<u>Генерация простых чисел для DSA</u>
 <u>Вероятностный тест на простоту</u>
 <u>Генерация простых чисел</u>
<u>Генерация случайных чисел для DSA</u>
 <u>Алгоритм для вычисления <i>m</i> значений числа <i>x</i></u>
 <u>Алгоритм для предварительного вычисления <i>k</i> и <i>r</i></u>
 <u>Создание функции <i>G</i> при помощи SHA</u>
 <u>Создание функции <i>G</i> при помощи DES</u>
<u>Генерация других параметров</u>
<u>Пример DSA</u>
<u>Примечания</u>
<u>Ссылки</u>
 <u>Зарубежные</u>
 <u>Русские</u>
 <u>Реализация</u>

Введение

Когда получено сообщение, получатель может пожелать проверить, не изменили ли это сообщение при его передаче. Получатель также может захотеть проверить подлинность подписавшейся стороны. DSA дает возможность сделать это.

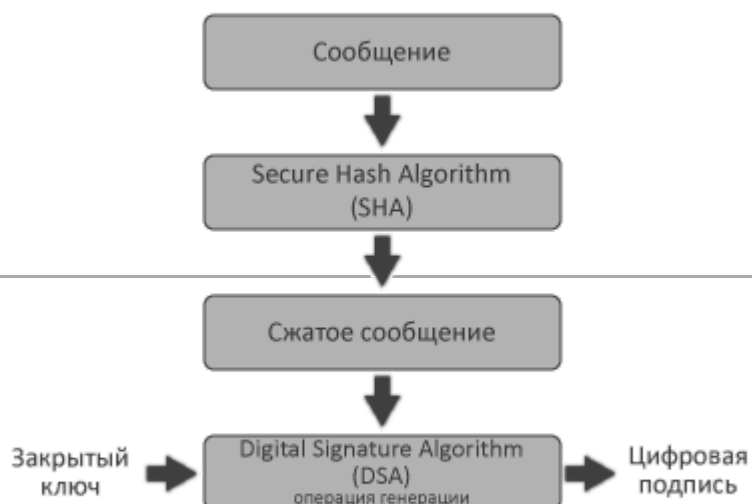
Использование DSA

DSA используется одной стороной для генерации подписи данных, а другой для проверки подлинности подписчика. Подпись генерируется при помощи закрытого ключа. Любая сторона может проверить подлинность цифровой подписи при помощи открытого ключа. Открытый ключ посылается вместе с подписанными данными. Открытый и закрытый ключи не совпадают.

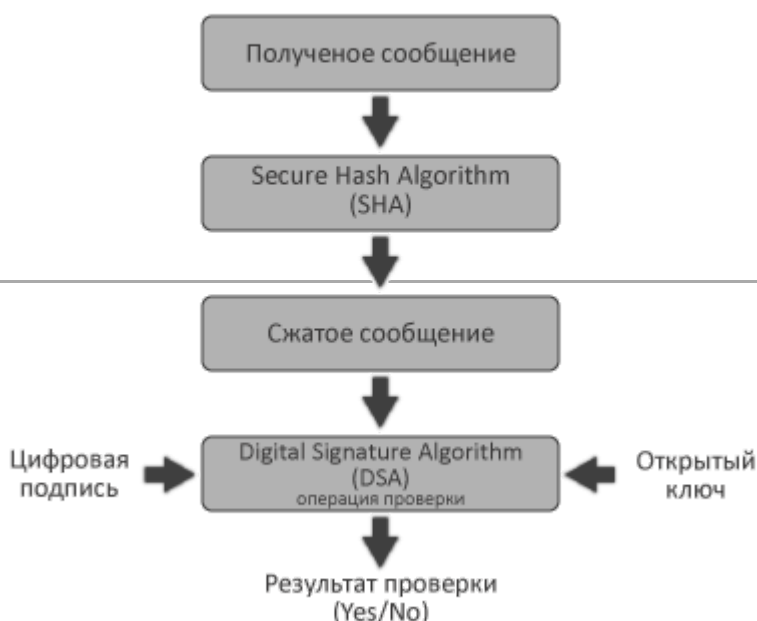
При генерации подписи для получения сжатой версии данных используется хеш-функция. Полученные данные обрабатываются при помощи DSA для получения цифровой подписи. Для проверки подписи используется та же хеш-функция. Хеш-функция описана в SHS (Secure Hash Standard).

Использование SHA вместе с DSA

Генерация подписи



Проверка подписи



Параметры DSA

DSA использует следующие параметры:

1. p – простое число p , где $2^{L-1} < p < 2^L$, $512 \leq L \leq 1024$ и L кратно 64
2. q – простой делитель $p-1$, причем $2^{159} < q < 2^{160}$
3. $g = h^{(p-1)/q} \bmod p$, где h любое целое число $1 < h < p-1$ такое, что $h^{(p-1)/q} \bmod p > 1$
4. x – случайное или псевдослучайное целое число, где $0 < x < q$
5. $y = g^x \bmod p$
6. k – случайное или псевдослучайное целое число, где $0 < k < q$.

Целые p , q и g могут быть открытыми и могут быть общими для группы людей. x и y являются закрытым и открытым ключами, соответственно. Параметры x и k используются только для генерации подписи и должны держаться в секрете. Параметр k разный для каждой подписи.

Генерация подписи

Подписью сообщения M является пара чисел r и s , где

$$\begin{aligned} r &= (g^k \bmod p) \bmod q \\ s &= (k^{-1}(\text{SHA}(M) + xr)) \bmod q. \end{aligned}$$

$\text{SHA}(M)$ — 160-битная бинарная строка.

Если $r = 0$ или $s = 0$, должно быть сгенерировано новое k и вычислена новая подпись. Если подпись вычислялась правильно, вероятность того, что $r = 0$ или $s = 0$ очень мала.

Подпись вместе с сообщением пересылается получателю.

Проверка подписи

Числа p , q , g и открытый ключ находятся в открытом доступе.

Пусть M' , r' и s' — полученные версии M , r и s соответственно, и пусть y — открытый ключ. При проверке подписи сначала нужно посмотреть, выполняются ли следующие неравенства:

$$\begin{aligned} 0 &< r' < q \\ 0 &< s' < q. \end{aligned}$$

Если хотя бы одно неравенство не выполнено, подпись должна быть отвергнута. Если условия неравенств выполнены, производятся следующие вычисления:

$$\begin{aligned} w &= (s')^{-1} \bmod q \\ u1 &= ((\text{SHA}(M')w) \bmod q \\ u2 &= ((r')w) \bmod q \\ v &= (((g)^{u1} (y)^{u2}) \bmod p) \bmod q. \end{aligned}$$

Если $v = r'$, то подлинность подписи подтверждена.

Если $v \neq r'$, то сообщение могло быть изменено, сообщение могло быть неправильно подписано или сообщение могло быть подписано мошенником. В этом случае полученные данные следует рассматривать как поврежденные.

Генерация простых чисел для DSA

Этот раздел включает алгоритмы для генерации простых чисел p и q для DSA. В этих алгоритмах используется генератор случайных чисел.

Вероятностный тест на простоту

Для генерации простых p и q необходим тест на простоту. Есть несколько быстрых вероятностных тестов. В нашем случае будет использоваться упрощенная версия теста Миллера-Рабина. Если повторить тест n раз, он выдаст простое число с вероятностью ошибки не больше $1/4^n$. Для проверки целого числа на простоту нужно:

```
Шаг 1. Устанавливаем  $i = 1$  и выбираем  $n \geq 50$ .
Шаг 2. Приравниваем  $w$  тестируемому числу и представляем его в виде  $w = 1 + 2^a m$ , где  $m$  — нечетное число.
Шаг 3. Генерируем случайное число  $b$ :  $1 < b < w$ .
Шаг 4. Устанавливаем  $j = 0$  и  $z = b^m \bmod w$ .
Шаг 5. Если  $j = 0$  и  $z = 1$ , или если  $z = w - 1$ , то переходим на шаг 9.
Шаг 6. Если  $j > 0$  и  $z = 1$ , то переходим на шаг 8.
Шаг 7.  $j = j + 1$ . Если  $j < a$ , то устанавливаем  $z = z^2 \bmod w$  и переходим на шаг 5.
Шаг 8.  $w$  не простое. Стоп.
Шаг 9. Если  $i < n$ , то устанавливаем  $i = i + 1$  и переходим на шаг 3. Иначе, возможно  $w$  — простое число.
```

Генерация простых чисел

Для DSS нужны 2 простых числа p и q , которые должны удовлетворять следующим условиям:

```
 $2^{159} < q < 2^{160}$ 
 $2^{L-1} < p < 2^L$ , где  $L = 512 + 64j$ , причем  $0 \leq j < 8$ 
 $p - 1$  делится на  $q$ .
```

Для генерации простого q : $2^{159} < q < 2^{160}$ используется SHA-1 и начальное число SEED. После этого число SEED используется для создания числа X : $2^{L-1} < X < 2^L$. Простое p получается округлением X таким образом, чтобы полученное число было равно $1 \bmod 2q$.

Пусть $L - 1 = n * 160 + b$, где b и n целые и принимают значения от 0 до 160.

```
Шаг 1. Выбираем произвольную последовательность из как минимум 160 бит и называем её SEED. Пусть  $g$  — длина SEED в битах.
Шаг 2. Вычисляем  $U = \text{SHA}[\text{SEED}] \text{ XOR } \text{SHA}[(\text{SEED}+1) \bmod 2^g]$ .
Шаг 3. Создаем  $q$  из  $U$  устанавливая младший и старший бит равным 1:
 $q = U \text{ OR } 2^{159} \text{ OR } 1$ .
Заметим, что  $2^{159} < q < 2^{160}$ .
Шаг 4. Проверяем  $q$  на простоту.
Шаг 5. Если  $q$  непростое, переходим на шаг 1.
Шаг 6. Пусть  $\text{counter} = 0$  и  $\text{offset} = 2$ .
Шаг 7. Для  $k = 0, \dots, n$  вычисляем  $V_k = \text{SHA}[(\text{SEED} + \text{offset} + k) \bmod 2^g]$ .
Шаг 8. Вычисляем  $W = V_0 + V_1 * 2^{160} + \dots + V_{n-1} * 2^{(n-1)*160} + (V_n \bmod 2^b) * 2^{n*160}$ 
 $X = W + 2^{L-1}$ .
Заметим, что  $0 \leq W < 2^{L-1}$  и  $2^{L-1} < X < 2^L$ .
Шаг 9. Пусть  $c = X \bmod 2q$  и  $p = X - (c - 1)$ . Заметим, что  $p$  равно  $1 \bmod 2q$ .
Шаг 10. Если  $p < 2^{L-1}$ , тогда переходим на шаг 13.
Шаг 11. Проверяем  $p$  на простоту.
Шаг 12. Если  $p$  прошло тест на 11 шаге, переходим на 15 шаг.
Шаг 13.  $\text{counter} = \text{counter} + 1$  и  $\text{offset} = \text{offset} + n + 1$ .
Шаг 14. Если  $\text{counter} \geq 2^{12} = 4096$  переходим на шаг 1, иначе переходим на шаг 7.
Шаг 15. Сохраняем SEED и counter для того, чтобы подтвердить правильную генерацию  $p$  и  $q$ .
```

Генерация случайных чисел для DSA

Для любой реализации DSA требуются случайные или псевдослучайные целые числа. Эти числа выбираются при помощи методов описанных в этом разделе или при помощи других одобренных FIPS методов.

Алгоритм в разделе 7.1 может быть использован для генерации x . Алгоритм для k и g описан в разделе 7.2. Алгоритмы используют одностороннюю функцию (функцию, обратное значение которой очень трудно вычислить) $G(t, c)$, где t имеет размер 160 бит, c имеет размер b бит ($160 < b < 512$) и $G(t, c)$ — 160 бит. G может быть создана с помощью Secure Hash Algorithm (SHA-1) или с помощью Data Encryption Standard (DES), описанные в разделах 7.3 и 7.4 соответственно.

Алгоритм для вычисления m значений числа x

Пусть x — секретный ключ подписывающей стороны. Следующий алгоритм можно использовать для генерации m значений числа x :

```
Шаг 1. Выбираем новое значение для исходного ключа, XKEY.
Шаг 2. В шестнадцатеричной системе счисления
t = 67452301 EFCDAB89 98BADCFE 10325476 C3D2E1F0.
Это начальное значение для H0 || H1 || H2 || H3 || H4 в SHA.
Шаг 3. Для j = 0..m - 1
  а. XSEEDj = опциональное значение, введенное пользователем.
  б. XVAL = (XKEY + XSEEDj) mod 2b.
  в. xj = G(t, XVAL) mod q.
  г. XKEY = (1 + XKEY + xj) mod 2b.
```

Алгоритм для предварительного вычисления k и g

Этот алгоритм может быть использован для предварительного вычисления k , k^{-1} и g для m сообщений одновременно. Алгоритм:

```
Шаг 1. Выбирается секретное начальное значение для KKEY.
Шаг 2. В шестнадцатеричной системе счисления
t = EFCDAB89 98BADCFE 10325476 C3D2E1F0 67452301.
Это начальный сдвиг для H0 || H1 || H2 || H3 || H4 в SHA.
Шаг 3. Для j = 0..m - 1:
  а. k = G(t, KKEY) mod q.
  б. Вычисляем kj-1 = k-1 mod q.
  в. Вычисляем rj = (gk mod p) mod q.
  г. KKEY = (1 + KKEY + k) mod 2b.
Шаг 4. Предположим M0, ..., Mm-1 - следующие m сообщений. Для j = 0..m - 1:
  а. Пусть h = SHA(Mj).
  б. sj = (kj-1(h + xrj)) mod q.
  в. rj, sj - подпись для Mj.
Шаг 5. t = h.
Шаг 6. Переходим на шаг 3.
```

Шаг 3 дает возможность вычислить величины, необходимые для подписи следующих m сообщений. Шаг 4 выполняется сразу после того, когда получены эти m сообщений.

Создание функции G при помощи SHA

$G(t, c)$ может быть получена при помощи SHA-1, но перед этим $\{H_j\}$ и M_1 должны быть инициализированы следующим образом:

```
1. Инициализируем {Hj} деление 160-битного значения t на пять 32-битных сегмента:
t = t0 || t1 || t2 || t3 || t4
Тогда Hj = tj для j = 0..4.
2. Блок сообщения M1 определяется следующим образом:
```

$$M_1 = c || 0^{512-b}$$

(Первые b бит сообщения M_1 содержат c , а оставшиеся $(512-b)$ бит устанавливаются нулями).

После этого выполняется SHA-1^[1] и получаем 160-битную строку $G(t, c)$, представленную в виде:

$$H_0 || H_1 || H_2 || H_3 || H_4.$$

Создание функции G при помощи DES

Пусть $a \text{ XOR } b$ обозначает побитовое исключающее «ИЛИ» (сложение по модулю 2). Пусть a_1, a_2, b_1, b_2 — 32-строки. Пусть b_1' — 24 младших бита числа b_1 . Пусть $K = b_1' || b_2$ и $A = a_1 || a_2$. Обозначим

$$DES_{b_1, b_2}(a_1, a_2) = DES_K(A)$$

$DES_K(A)$ обозначает обычное DES-шифрование^[2] 64-битного блока A при помощи 56-битного ключа K . Предположим, что t и c имеют размер 160 бит каждое. Для вычисления $G(t, c)$:

```
Шаг 1. Записываем
t = t1 || t2 || t3 || t4 || t5
c = c1 || c2 || c3 || c4 || c5
Каждое ti и ci имеет размер 32 бита.
Шаг 2. Для i = 1..5:
xi = ti XOR ci
Шаг 3. Для i = 1..5:
b1 = c((i+3) mod 5) + 1
b2 = c((i+2) mod 5) + 1
a1 = xi
a2 = x(i mod 5) + 1 XOR x((i+3) mod 5) + 1
yi,1 || yi,2 = DESb1, b2(a1, a2) (yi,1, yi,2 = 32 бита)
Шаг 4. Для i = 1..5:
zi = yi,1 XOR y((i+1) mod 5)+1,2 XOR y((i+2) mod 5)+1,1
Шаг 5. G(t, c) = z1 || z2 || z3 || z4 || z5
```

Генерация других параметров

В этом разделе приведены алгоритмы для генерации g , k^{-1} и s^{-1} , которые используются в DSS. Для генерации g :

```
Шаг 1. Генерация p и q описана выше.
Шаг 2. Пусть e = (p - 1)/q.
Шаг 3. Приравниваем h любому целому числу: 1 < h < p - 1.
Шаг 4. g = he mod p.
Шаг 5. Если g = 1, переходим на шаг 3.
```

Для вычисления $n^{-1} \bmod q$, где $0 < n < q$ и $0 < n^{-1} < q$:

```
Шаг 1. i = q, h = n, v = 0 и d = 1.
Шаг 2. Пусть t = i DIV h, где DIV - целочисленное деление.
Шаг 3. x = h.
Шаг 4. h = i - tx.
Шаг 5. i = x.
Шаг 6. x = d.
Шаг 7. d = v - tx.
```

Шаг 8. $v = x$.

Шаг 9. Если $h > 0$, переходим на шаг 2.

Шаг 10. Пусть $n^{-1} = v \bmod q$.

Заметим, что на шаге 10, v может быть отрицательным.

Пример DSA

Пусть $L = 512$ (размер p). В этом примере все величины будут в шестнадцатеричном представлении. Величины p и q были сгенерированы, как описано выше, используя следующее 160-битное значение SEED:

SEED = d5014e4b 60ef2ba8 b6211b40 62ba3224 e0427dd3

С этим SEED, алгоритм нашел p и q в момент, когда counter = 105. x было сгенерировано при помощи алгоритма, описанного в разделе 7.1, с использованием SHA-1 для генерации G (раздел 7.3) 160-битный XKEY:

XKEY = bd029bbe 7f51960b cf9edb2b 61f06f0f eb5a38b6

t = 67452301 EFCDAB89 98BADCFE 10325476 C3D2E1F0

$x = G(t, XKEY) \bmod q$

k было сгенерировано как описано в разделе 7.2 с использованием SHA-1 для генерации G (раздел 7.3) 160-битный KKEY:

KKEY = 687a66d9 0648f993 867e121f 4ddf9ddb 01205584

t = EFCDAB89 98BADCFE 10325476 C3D2E1F0 67452301

$k = G(t, KKEY) \bmod q$

Окончательно:

h = 2

p = 8df2a494 492276aa 3d25759b b06869cb eac0d83a fb8d0cf7 cbb8324f 0d7882e5
d0762fc5 b7210eaf c2e9adac 32ab7aac 49693dfb f83724c2 ec0736ee 31c80291

q = c773218c 737ec8ee 993b4f2d ed30f48e dace915f

g = 626d0278 39ea0a13 413163a5 5b4cb500 299d5522 956cefcf 3bff10f3 99ce2c2e
71cb9de5 fa24babf 58e5b795 21925c9c c42e9f6f 464b088c c572af53 e6d78802

x = 2070b322 3dba372f de1c0ffc 7b2e3b49 8b260614

k = 358dad57 1462710f 50e254cf 1a376b2b deaadfbf

$k^{-1} = 0d516729 8202e49b 4116ac10 4fc3f415 ae52f917$

M = слово «abc» из английского алфавита(ASCII)

(SHA-1)(M) = a9993e36 4706816a ba3e2571 7850c26c 9cd0d89d

y = 19131871 d75b1612 a819f29d 78d1b0d7 346f7aa7 7bb62a85 9bfd6c56 75da9d21
2d3a36ef 1672ef66 0b8c7c25 5cc0ec74 858fba33 f44c0669 9630a76b 030ee333

r = 8bac1ab6 6410435c b7181f95 b16ab97c 92b341c0

s = 41e2345f 1f56df24 58f426d1 55b4ba2d b6dcd8c8


```

w = 9df4ece5 826be95f ed406d41 b43edc0b 1c18841b
u1 = bf655bd0 46f0b35e c791b004 804afcbb 8ef7d69d
u2 = 821a9263 12e97ade abcc8d08 2b527897 8a2df4b0
gu1 mod p = 51b1bf86 7888e5f3 af6fb476 9dd016bc fe667a65 aafc2753 9063bd3d 2b138b4c
              e02cc0c0 2ec62bb6 7306c63e 4db95bbf 6f96662a 1987a21b e4ec1071 010b6069
yu2 mod p = 8b510071 2957e950 50d6b8fd 376a668e 4b0d633c 1e46e665 5c611a72 e2b28483
              be52c74d 4b30de61 a668966e dc307a67 c19441f4 22bf3c34 08aeba1f 0a4dbec7
v = 8bac1ab6 6410435c b7181f95 b16ab97c 92b341c0

```

Примечания

1. FIPS PUB 180-1 (<https://www.webcitation.org/66kq6hVtl?url=http://www.itl.nist.gov/fipspubs/fip180-1.htm>) (англ.). — описание стандарта SHS. Архивировано из оригинала (<http://www.itl.nist.gov/fipspubs/fip180-1.htm>) 7 апреля 2012 года.
2. FIPS PUB 46-3 (<https://www.webcitation.org/66kuhPhSB?url=http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf>) (англ.). — описание стандарта DES. Архивировано из оригинала (<http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf>) 7 апреля 2012 года.

Ссылки

Зарубежные

- FIPS-186 (<https://www.webcitation.org/66kuhq8Pb?url=http://www.itl.nist.gov/fipspubs/fip186.htm>) (англ.). — the first version of the official DSA specification. Архивировано из оригинала (<http://www.itl.nist.gov/fipspubs/fip186.htm>) 7 апреля 2012 года.
- FIPS-186, change notice No.1 (<https://www.webcitation.org/66kuiFfkN?url=http://www.itl.nist.gov/fipspubs/186chg-1.htm>) (англ.). — the first change notice to the first version of the specification. Архивировано из оригинала (<http://www.itl.nist.gov/fipspubs/186chg-1.htm>) 7 апреля 2012 года.
- FIPS-186-1 (<https://www.webcitation.org/66kuieUsU?url=http://www.mozilla.org/projects/security/pki/nss/fips1861.pdf>) (англ.). — the first revision to the official DSA specification. Архивировано из оригинала (<http://www.mozilla.org/projects/security/pki/nss/fips1861.pdf>) 7 апреля 2012 года.
- FIPS-186-3 (https://www.webcitation.org/66kuj5Zrp?url=http://csrc.nist.gov/publications/fips/fips186-3/fips_186-3.pdf) (англ.). — the third revision to the official DSA specification. Архивировано из оригинала (http://csrc.nist.gov/publications/fips/fips186-3/fips_186-3.pdf) 7 апреля 2012 года.
- FIPS-186-4 (<http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf>) (англ.). — the fourth and current revision to the official DSA specification. Архивировано (<https://web.archive.org/web/20161227093019/http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf>) 27 декабря 2016 года.
- FIPS-186-3 Approval (https://www.webcitation.org/66kujVMH7?url=http://csrc.nist.gov/publications/fips/fips186-3/frn-fips_186-3.pdf) (англ.). — approval announcement of the third revision to the official DSA specification. Архивировано из оригинала (http://csrc.nist.gov/publications/fips/fips186-3/frn-fips_186-3.pdf) 7 апреля 2012 года.
- Recommendation for Key Management -- Part 1: general (<http://csrc.nist.gov/publications/nistpubs/800-57/SP800-57-Part1.pdf>) (англ.). — NIST Special Publication 800-57, p. 62–63.

Архивировано (<https://web.archive.org/web/20050821162420/http://csrc.nist.gov/publications/nistpubs/800-57/SP800-57-Part1.pdf>) 21 августа 2005 года.

Русские

- Алгоритм DSA (<https://web.archive.org/web/20130106100817/http://www.wincrypt.chat.ru/dsa.htm>). — краткое описание алгоритма DSA. Архивировано из оригинала (<http://wincrypt.chat.ru/dsa.htm>) 6 января 2013 года.

Реализация

- DSA-методы (https://www.webcitation.org/66kukcMN5?url=http://msdn.microsoft.com/ru-ru/library/system.security.cryptography.dsa_methods.aspx). — описание методов DSA-класса из библиотеки классов платформы .NET Framework. Архивировано из оригинала (http://msdn.microsoft.com/ru-ru/library/system.security.cryptography.dsa_methods.aspx) 7 апреля 2012 года.

Источник — https://ru.wikipedia.org/w/index.php?title=Digital_Signature_Standard&oldid=123377751

Эта страница в последний раз была отредактирована 18 июня 2022 в 17:09.

Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия.

Wikipedia® — зарегистрированный товарный знак некоммерческой организации Фонд Викимедиа (Wikimedia Foundation, Inc.)